Percurso em Profundidade

Prof. Carlos Vinicius
(vinicius.lima@ufca.edu.br)

Algoritmos em Grafos

Universidade Federal do Cariri – UFCA
(2026/02)

Sumário

1. Introdução
2. Digrafos Acíclicos
3. Busca em Profundidade – DFS
4. Ordenação Topológica

Ordenação de Vértices ou arestas

Ordenação de Vértices ou arestas

- Podemos ordenar vértices ou arestas segundo diferentes critérios:

Ordenação de Vértices ou arestas

- Podemos ordenar vértices ou arestas segundo diferentes critérios:
  - Ordenação segundo seus **pesos**.

Ordenação de Vértices ou arestas

- Podemos ordenar vértices ou arestas segundo diferentes critérios:
  - Ordenação segundo seus **pesos**.
  - Ordenação de vértices pelos graus.

Ordenação de Vértices ou arestas

- Podemos ordenar vértices ou arestas segundo diferentes critérios:
  - Ordenação segundo seus **pesos**.
  - Ordenação de vértices pelos graus.
  - Ordenar as listas de adjacências segundo os graus.

Ordenação de Vértices ou arestas

- Podemos ordenar vértices ou arestas segundo diferentes critérios:
  - Ordenação segundo seus **pesos**.
  - Ordenação de vértices pelos graus.
  - Ordenar as listas de adjacências segundo os graus.
  - Ordenação topológica.

Ordenação de Vértices ou arestas

- Podemos ordenar vértices ou arestas segundo diferentes critérios:
  - Ordenação segundo seus **pesos**.
  - Ordenação de vértices pelos graus.
  - Ordenar as listas de adjacências segundo os graus.
  - Ordenação topológica.
- Utiliza-se com frequência o método de **Ordenação por Caixas** para os graus.

Ordenação de Vértices ou arestas

- Podemos ordenar vértices ou arestas segundo diferentes critérios:
  - Ordenação segundo seus **pesos**.
  - Ordenação de vértices pelos graus.
  - Ordenar as listas de adjacências segundo os graus.
  - Ordenação topológica.
- Utiliza-se com frequência o método de **Ordenação por Caixas** para os graus.
- Este método possui complexidade $O(n + m)$.

Sumário

1. Introdução
2. Digrafos Acíclicos
3. Busca em Profundidade – DFS
4. Ordenação Topológica

Digrafos Acíclicos

- Seja $G = (V, E)$ um Grafo Acíclico Direcionado (DAG).

Digrafos Acíclicos

- Seja $G = (V, E)$ um Grafo Acíclico Direcionado (DAG).
- DAG's representam precedências entre eventos, por exemplo.

Digrafos Acíclicos

- Seja $G = (V, E)$ um Grafo Acíclico Direcionado (DAG).
- DAG's representam precedências entre eventos, por exemplo.

Teorema: Todo DAG $G = (V, E)$ possui ao menos uma fonte e um sumidouro.

Digrafos Acíclicos

- Seja $G = (V, E)$ um Grafo Acíclico Direcionado (DAG).
- DAG's representam precedências entre eventos, por exemplo.

Teorema: Todo DAG $G = (V, E)$ possui ao menos uma fonte e um sumidouro.

- Suponha por contradição que G não possui sumidouros.

Digrafos Acíclicos

- Seja $G = (V, E)$ um Grafo Acíclico Direcionado (DAG).
- DAG's representam precedências entre eventos, por exemplo.

Teorema: Todo DAG $G = (V, E)$ possui ao menos uma fonte e um sumidouro.

- Suponha por contradição que G não possui sumidouros.
- Logo o grau de saída de todos os vértices é positivo.

Digrafos Acíclicos

- Seja $G = (V, E)$ um Grafo Acíclico Direcionado (DAG).
- DAG's representam precedências entre eventos, por exemplo.

Teorema: Todo DAG $G = (V, E)$ possui ao menos uma fonte e um sumidouro.

- Suponha por contradição que G não possui sumidouros.
- Logo o grau de saída de todos os vértices é positivo.
- Seja $v \in V(G)$. Escolha $u \in N^+(v)$ e percorra a aresta vu.

Digrafos Acíclicos

- Seja $G = (V, E)$ um Grafo Acíclico Direcionado (DAG).
- DAG's representam precedências entre eventos, por exemplo.

Teorema: Todo DAG $G = (V, E)$ possui ao menos uma fonte e um sumidouro.

- Suponha por contradição que G não possui sumidouros.
- Logo o grau de saída de todos os vértices é positivo.
- Seja $v \in V(G)$. Escolha $u \in N^+(v)$ e percorra a aresta vu.
- Continue este processo a partir de u enquanto for possível.

Digrafos Acíclicos

- Seja $G = (V, E)$ um Grafo Acíclico Direcionado (DAG).
- DAG's representam precedências entre eventos, por exemplo.

Teorema: Todo DAG $G = (V, E)$ possui ao menos uma fonte e um sumidouro.

- Suponha por contradição que G não possui sumidouros.
- Logo o grau de saída de todos os vértices é positivo.
- Seja $v \in V(G)$. Escolha $u \in N^+(v)$ e percorra a aresta vu.
- Continue este processo a partir de u enquanto for possível.
- Como G é finito, retornaremos a um vértice já visitado neste percurso, formando assim um ciclo direcionado, uma contradição.

Digrafos Acíclicos

- Seja $G = (V, E)$ um Grafo Acíclico Direcionado (DAG).
- DAG's representam precedências entre eventos, por exemplo.

Teorema: Todo DAG $G = (V, E)$ possui ao menos uma fonte e um sumidouro.

- Suponha por contradição que G não possui sumidouros.
- Logo o grau de saída de todos os vértices é positivo.
- Seja $v \in V(G)$. Escolha $u \in N^+(v)$ e percorra a aresta vu.
- Continue este processo a partir de u enquanto for possível.
- Como G é finito, retornaremos a um vértice já visitado neste percurso, formando assim um ciclo direcionado, uma contradição.
- Para fontes podemos usar argumento análogo percorrendo vizinhos de entrada.

Ordem Parcial

Ordem Parcial

- Dado um DAG $G = (V, E)$, o **fecho transitivo** de D é o digrafo $D_f(V, E_f)$ maximal obtido de G que preserva a alcançabilidade.

Ordem Parcial

- Dado um DAG $G = (V, E)$, o **fecho transitivo** de D é o digrafo $D_f(V, E_f)$ maximal obtido de G que preserva a alcançabilidade.
- Ou seja, para todo $u, v \in V$ tal que u alcança v em D, então $uv \in E_f$.

Ordem Parcial

- Dado um DAG $G = (V, E)$, o **fecho transitivo** de D é o digrafo $D_f(V, E_f)$ maximal obtido de G que preserva a alcançabilidade.
- Ou seja, para todo $u, v \in V$ tal que u alcança v em D, então $uv \in E_f$.
- Um **conjunto parcialmente ordenado** ou **ordem parcial** $(S, <)$ é definido por um conjunto não vazio S e uma relação binária $<$, tal que:
  - (i) $s_i \not< s_i$, para todo $s_i \in S$ (não reflexiva);
  - (ii) $s_i < s_j \rightarrow s_j \not< s_i$, para todo $s_i, s_j \in S$ (assimétrica);
  - (iii) $s_i < s_j$ e $s_j < s_w \rightarrow s_i < s_w$, para todo $s_i, s_j, s_w \in S$ (transitiva).

Ordem Parcial

- Dado um DAG $G = (V, E)$, o **fecho transitivo** de D é o digrafo $D_f(V, E_f)$ maximal obtido de G que preserva a alcançabilidade.
- Ou seja, para todo $u, v \in V$ tal que u alcança v em D, então $uv \in E_f$.
- Um **conjunto parcialmente ordenado** ou **ordem parcial** $(S, <)$ é definido por um conjunto não vazio S e uma relação binária $<$, tal que:
  - (i) $s_i \not< s_i$, para todo $s_i \in S$ (não reflexiva);
  - (ii) $s_i < s_j \rightarrow s_j \not< s_i$, para todo $s_i, s_j \in S$ (assimétrica);
  - (iii) $s_i < s_j$ e $s_j < s_w \rightarrow s_i < s_w$, para todo $s_i, s_j, s_w \in S$ (transitiva).
- Assim, o fecho transitivo $D_f(V, E_f)$ de um DAG $D = (V, E)$ é uma ordem parcial $(V, <)$.

Ordem Topológica

Ordem Topológica

- Podemos definir a seguinte ordem parcial $(V, <)$ a partir de um DAG $D = (V, E)$:

$$u < v \Leftrightarrow u \text{ alcança } v \text{ em } D, \text{ para todo } u, v \in V, u \neq v.$$

Ordem Topológica

- Podemos definir a seguinte ordem parcial $(V, <)$ a partir de um DAG $D = (V, E)$:

$$u < v \Leftrightarrow u \text{ alcança } v \text{ em } D, \text{ para todo } u, v \in V, u \neq v.$$

- Uma **ordenação topológica** $v_1, v_2, \ldots, v_n$ de V é definida como:

$$v_i < v_j \rightarrow i < j, \text{ para todo } 1 \leq i, j \leq n.$$

Ordem Topológica

- Podemos definir a seguinte ordem parcial $(V, <)$ a partir de um DAG $D = (V, E)$:

$$u < v \Leftrightarrow u \text{ alcança } v \text{ em } D, \text{ para todo } u, v \in V, u \neq v.$$

- Uma **ordenação topológica** $v_1, v_2, \ldots, v_n$ de V é definida como:

$$v_i < v_j \rightarrow i < j, \text{ para todo } 1 \leq i, j \leq n.$$

- Ou seja, é uma ordem dos vértices de modo que todas as arestas apontem da esquerda para a direita.

Ordem Topológica

- Podemos definir a seguinte ordem parcial $(V, <)$ a partir de um DAG $D = (V, E)$:

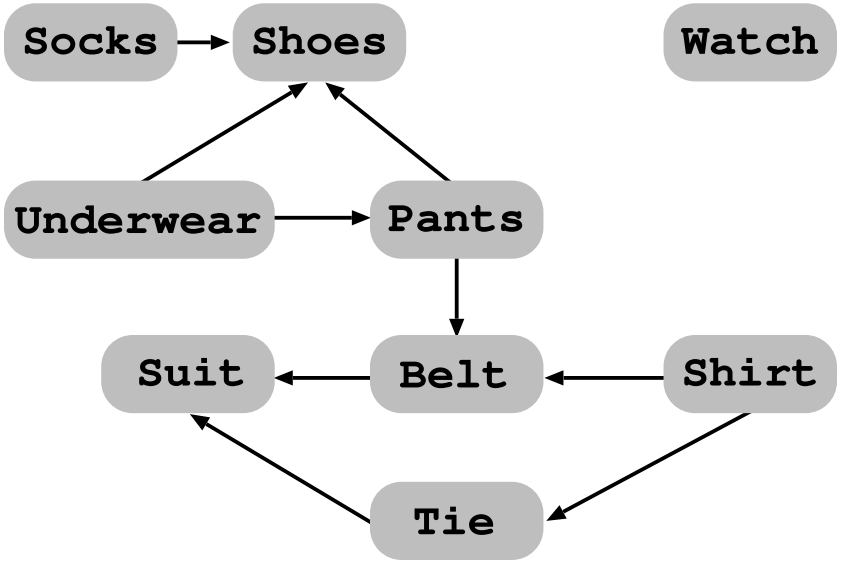
$$u < v \Leftrightarrow u \text{ alcança } v \text{ em } D, \text{ para todo } u, v \in V, u \neq v.$$

- Uma **ordenação topológica** $v_1, v_2, \ldots, v_n$ de V é definida como:

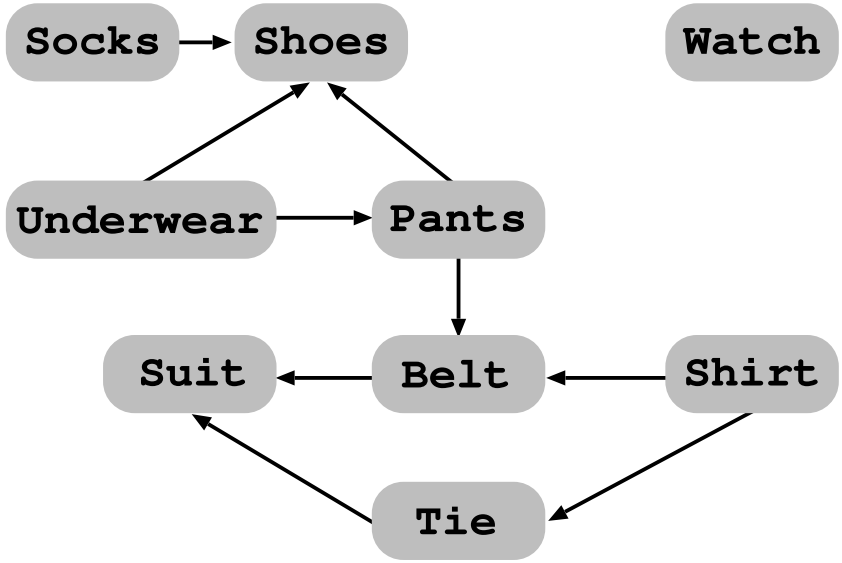
$$v_i < v_j \rightarrow i < j, \text{ para todo } 1 \leq i, j \leq n.$$

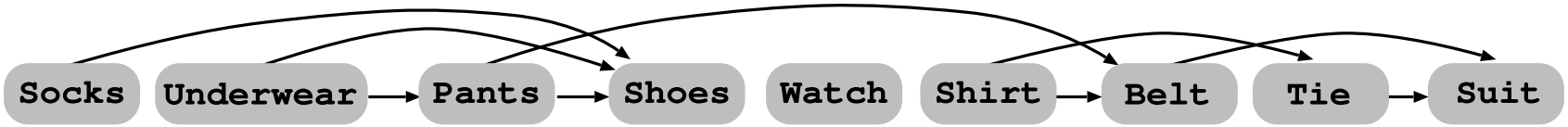
- Ou seja, é uma ordem dos vértices de modo que todas as arestas apontem da esquerda para a direita.
- Podem haver várias para o mesmo DAG.

Exemplo de Ordenação Topológica

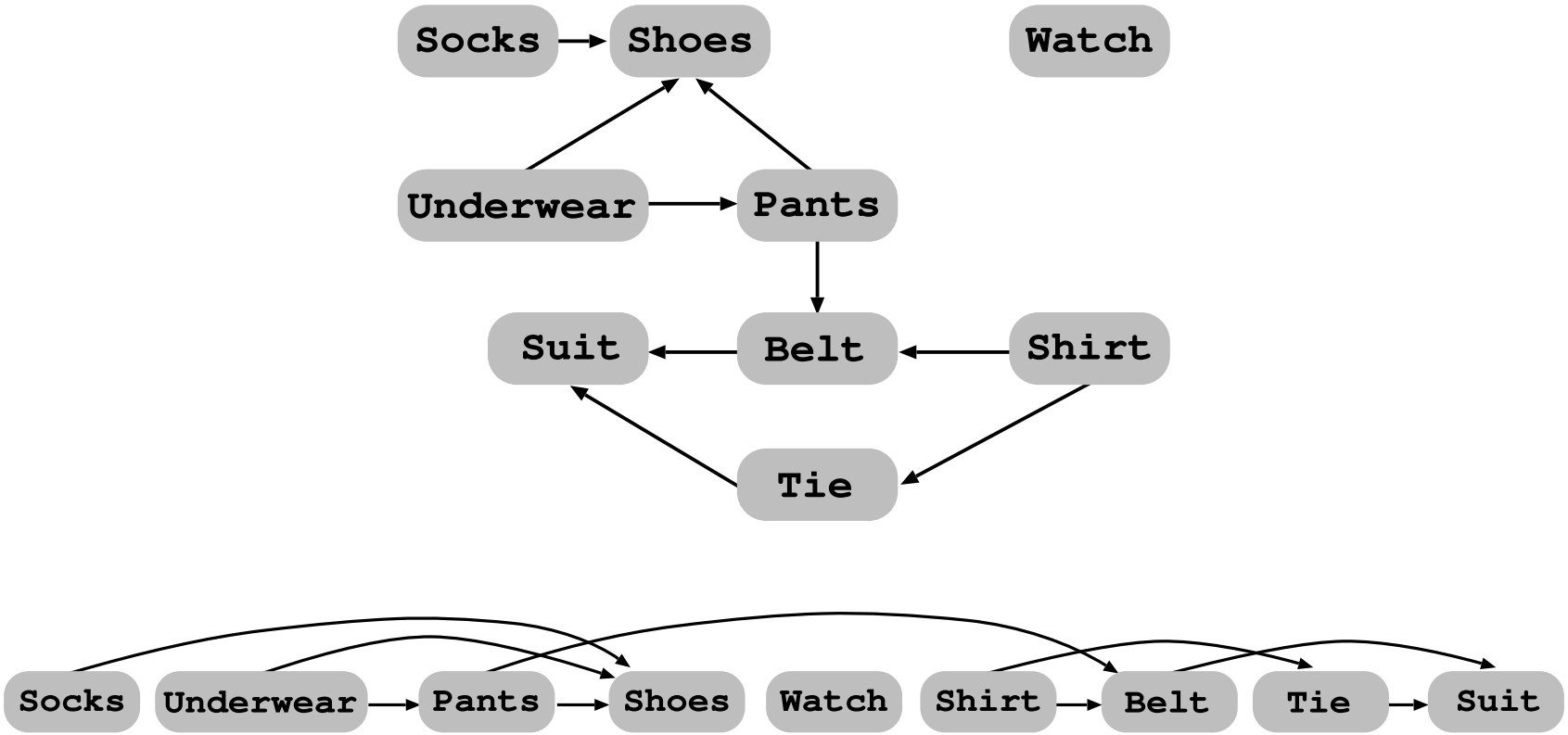


Exemplo de Ordenação Topológica





Exemplo de Ordenação Topológica

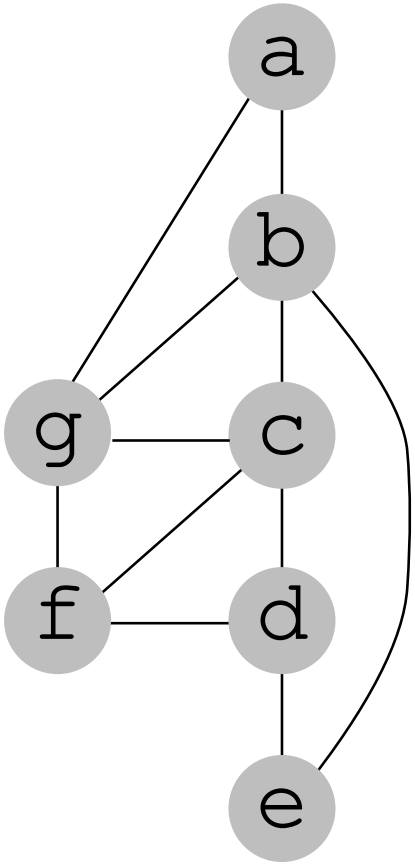


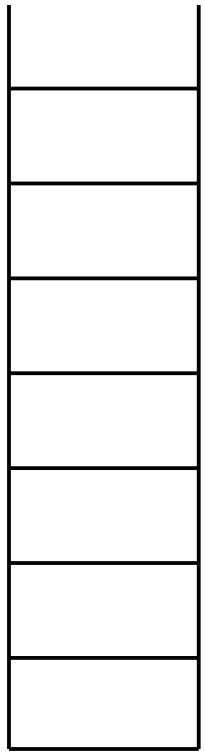
- Como encontrar uma ordenação topológica em um DAG?

Sumário

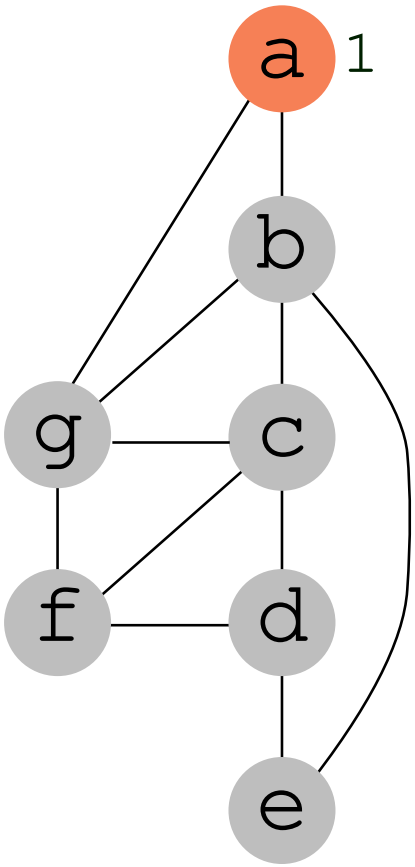
1. Introdução
2. Digrafos Acíclicos
3. Busca em Profundidade – DFS
4. Ordenação Topológica

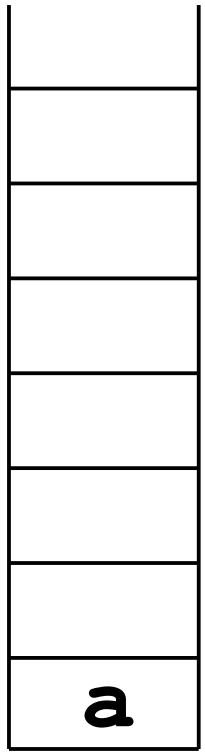
Busca em Profundidade (DFS – Depth-First Search)



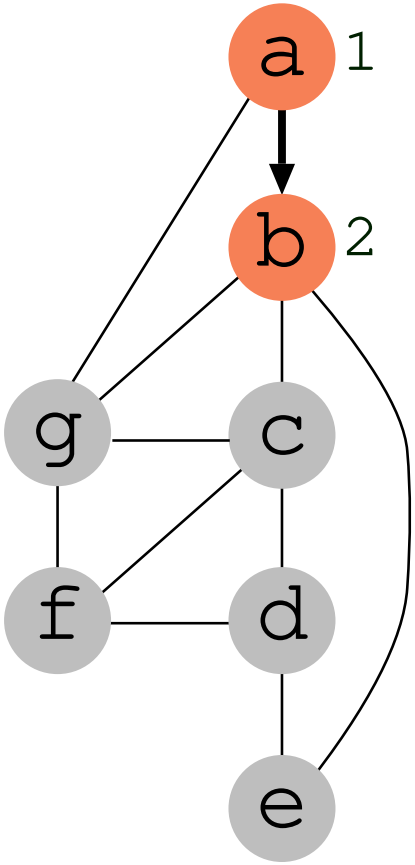


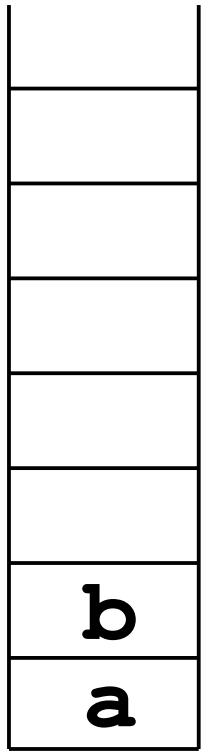
Busca em Profundidade (DFS – Depth-First Search)



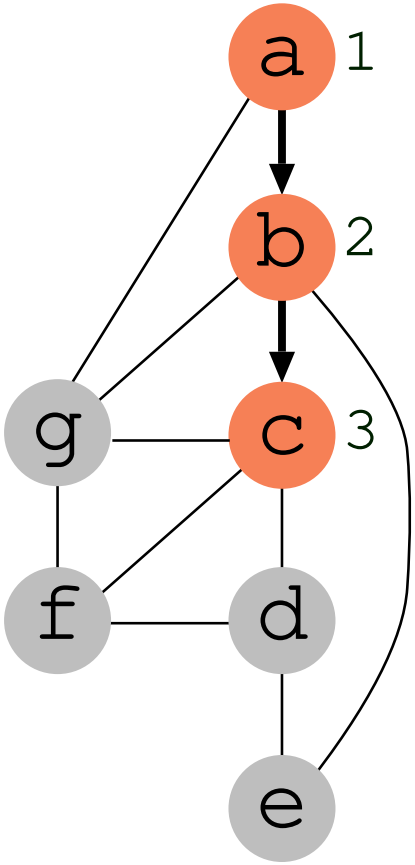


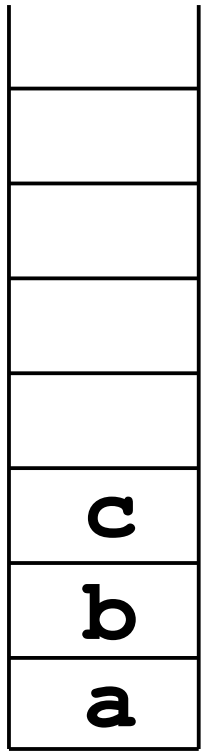
Busca em Profundidade (DFS – Depth-First Search)



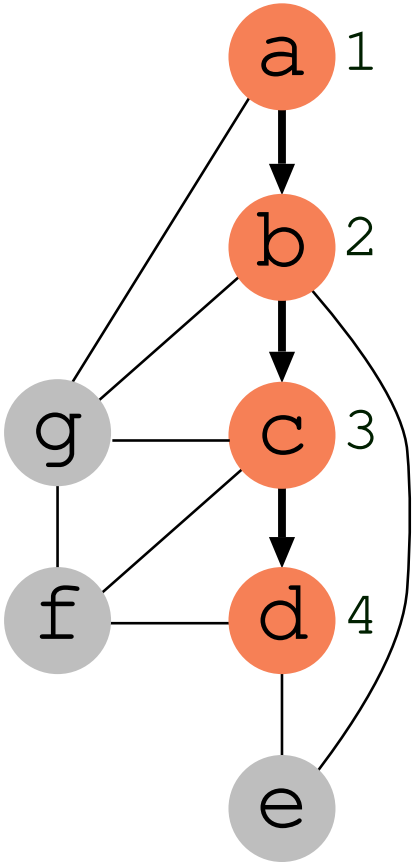


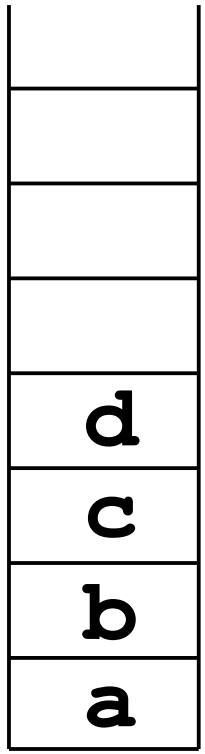
Busca em Profundidade (DFS – Depth-First Search)



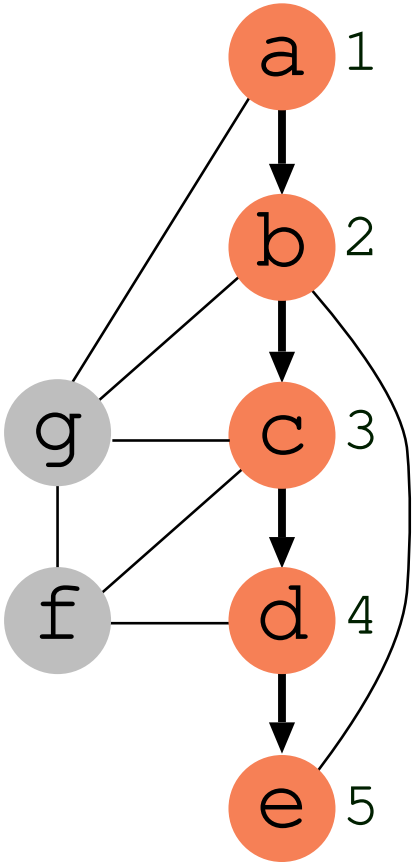


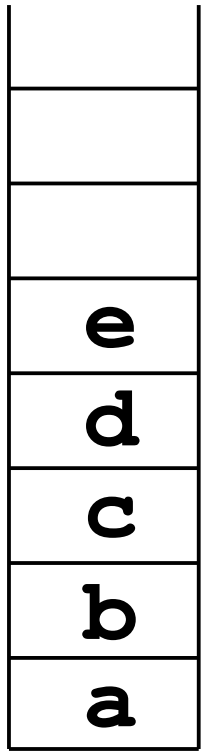
Busca em Profundidade (DFS – Depth-First Search)



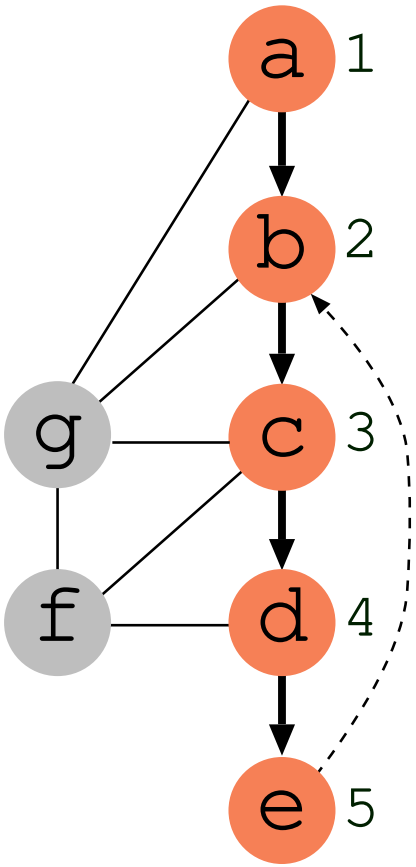


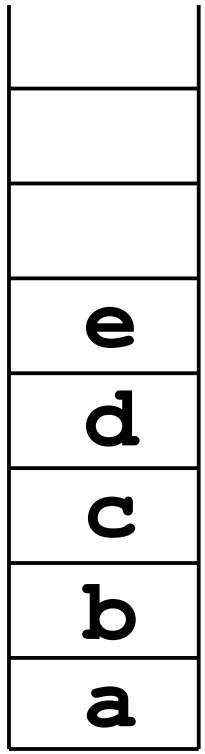
Busca em Profundidade (DFS – Depth-First Search)



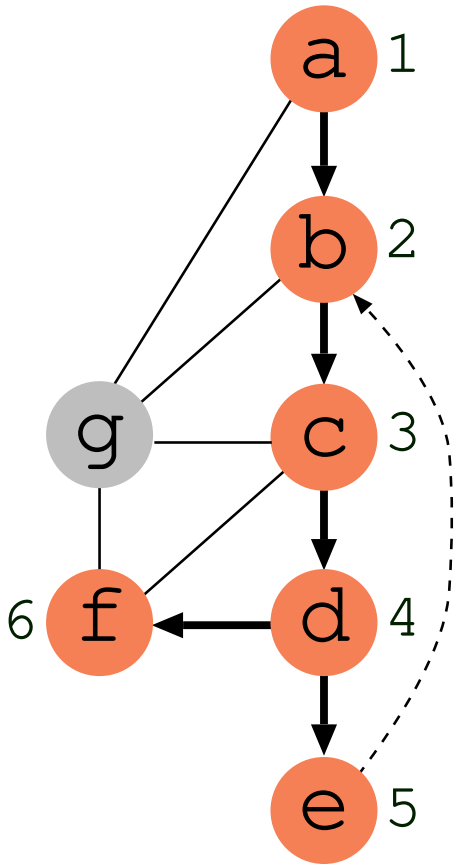


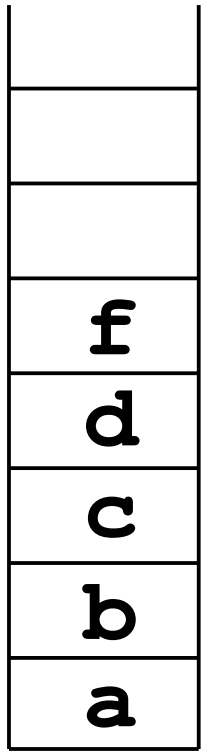
Busca em Profundidade (DFS – Depth-First Search)



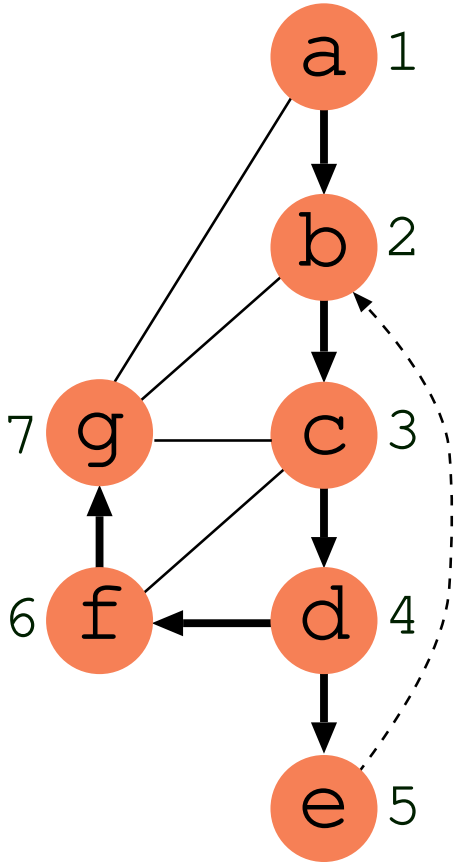


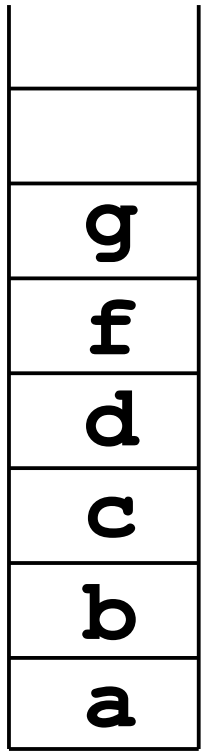
Busca em Profundidade (DFS – Depth-First Search)



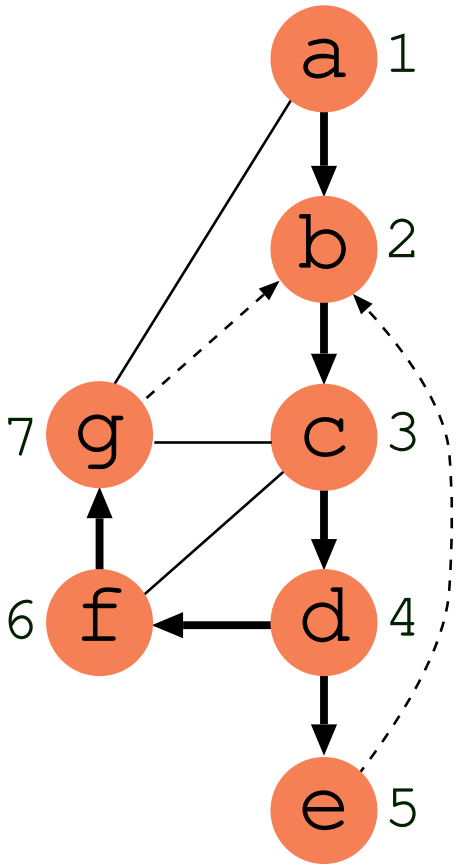


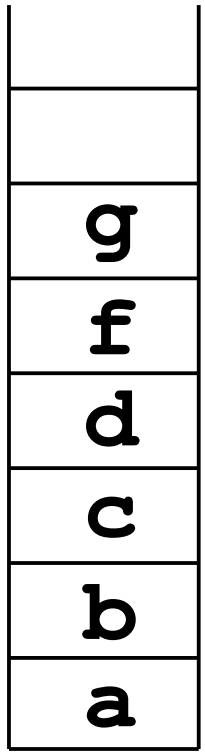
Busca em Profundidade (DFS – Depth-First Search)



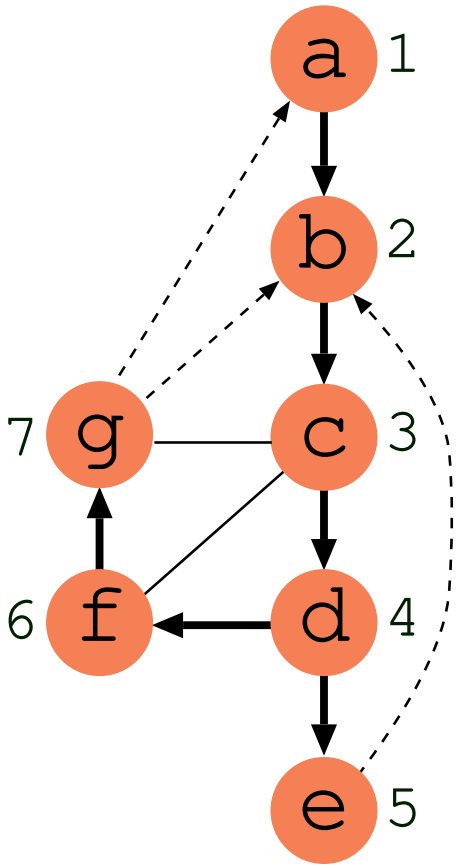


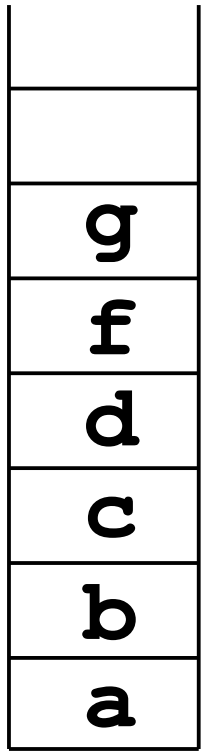
Busca em Profundidade (DFS – Depth-First Search)



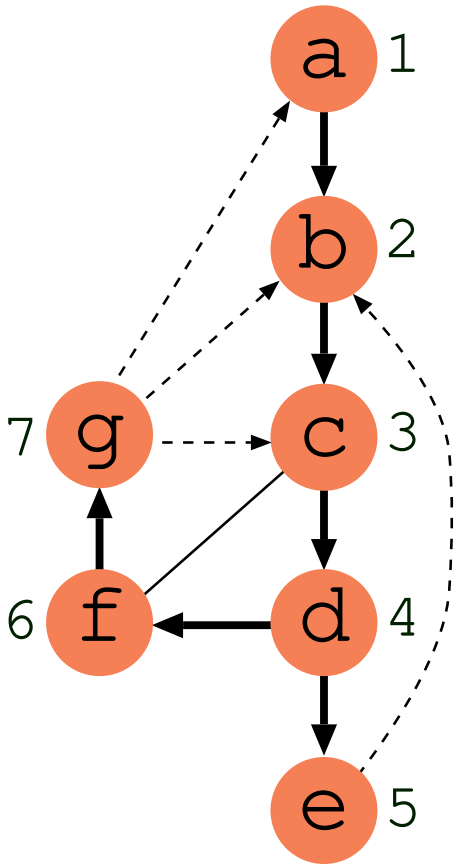


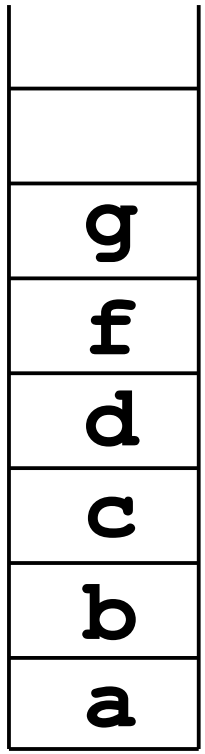
Busca em Profundidade (DFS – Depth-First Search)



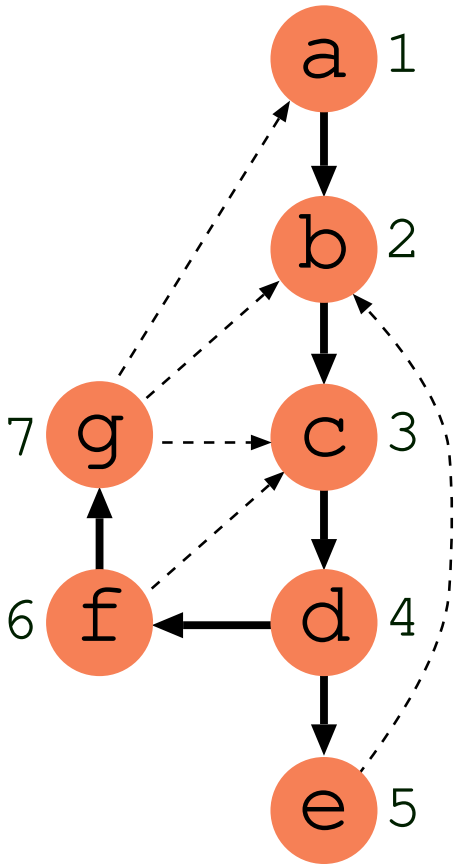


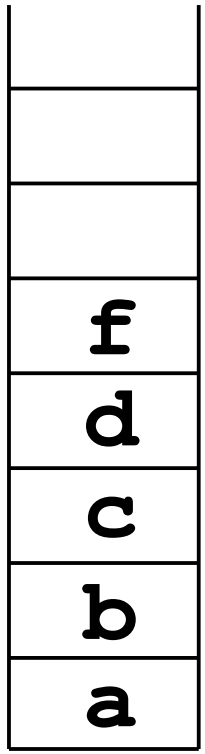
Busca em Profundidade (DFS – Depth-First Search)



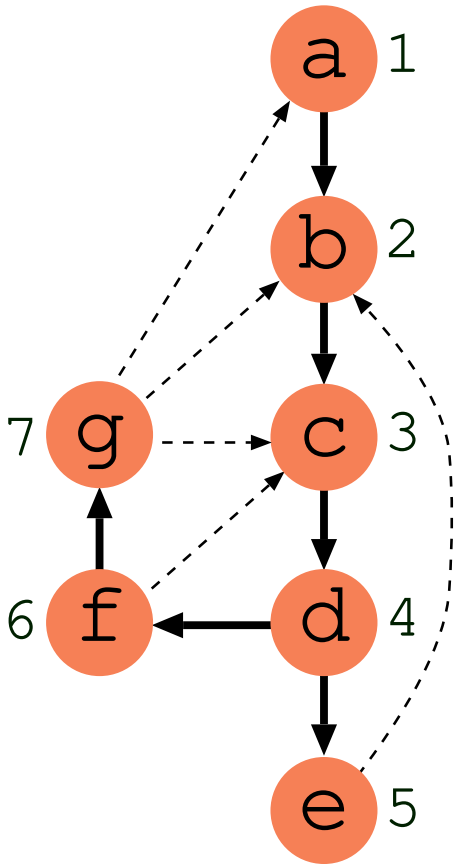


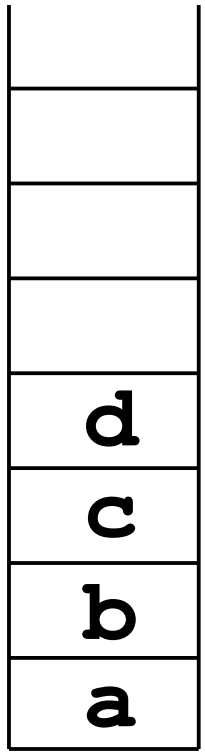
Busca em Profundidade (DFS – Depth-First Search)



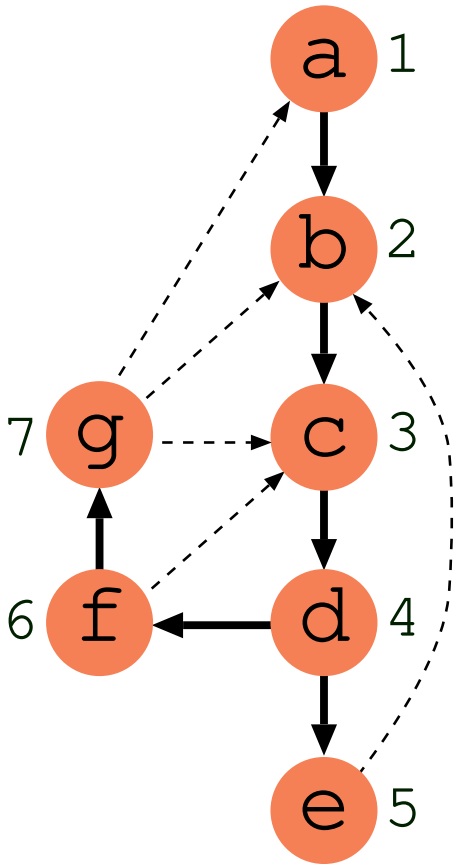


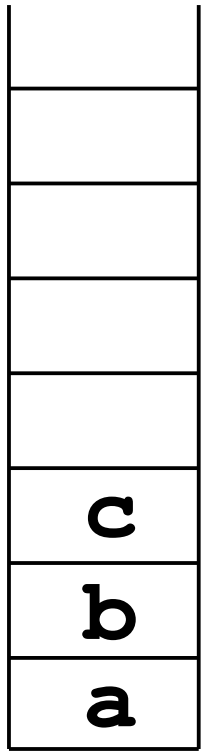
Busca em Profundidade (DFS – Depth-First Search)



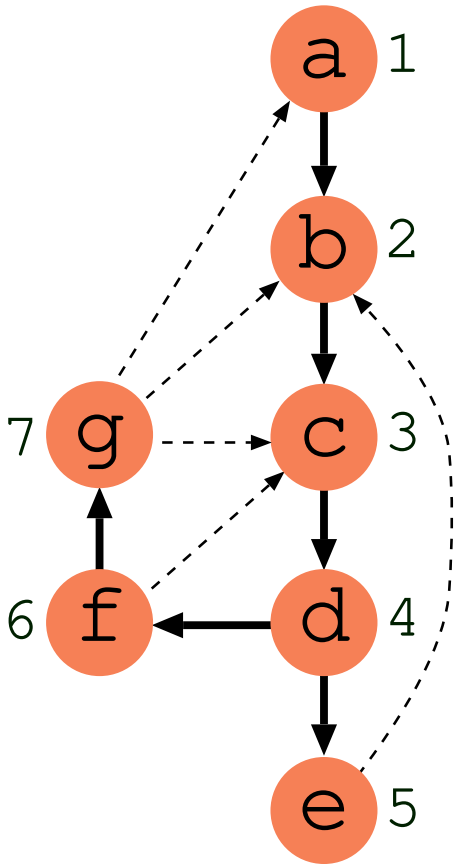


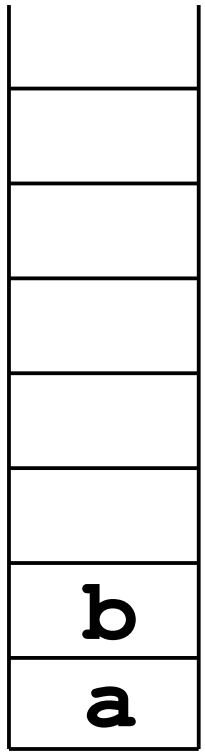
Busca em Profundidade (DFS – Depth-First Search)



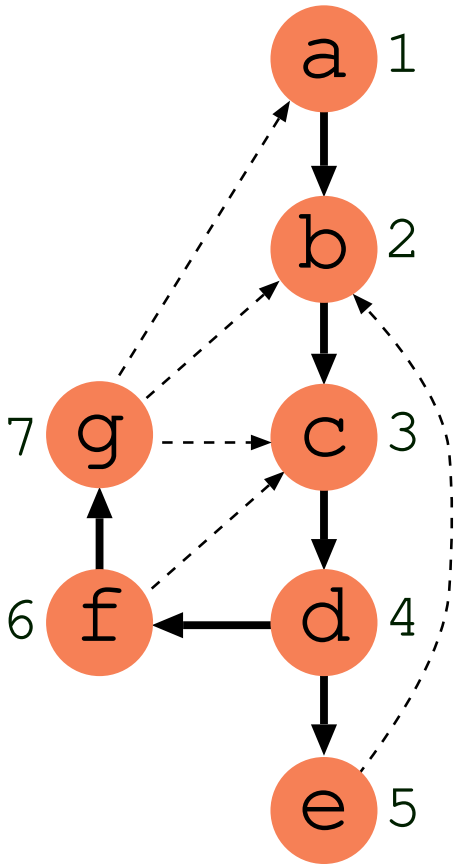


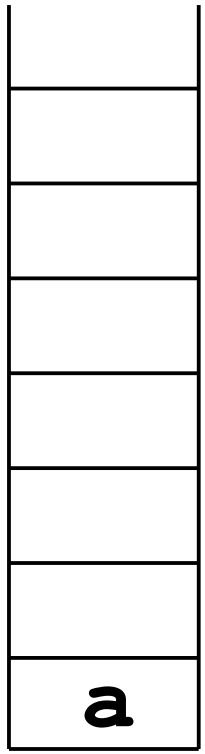
Busca em Profundidade (DFS – Depth-First Search)



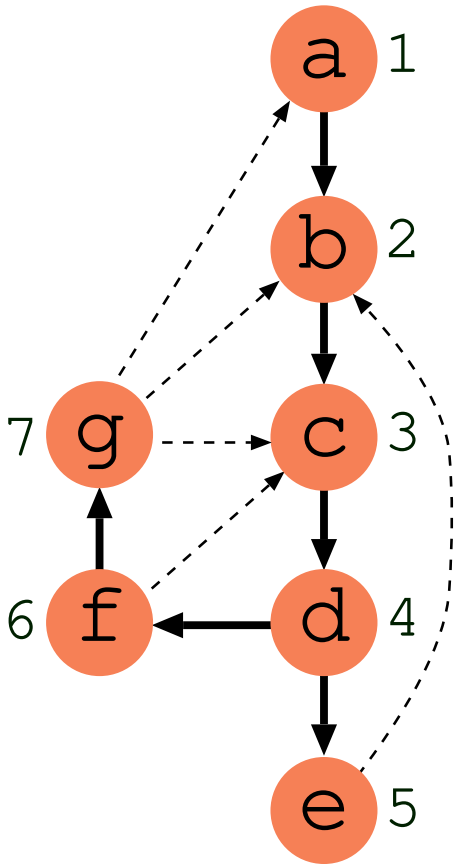


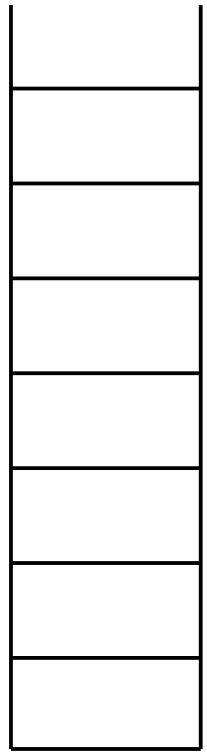
Busca em Profundidade (DFS – Depth-First Search)





Busca em Profundidade (DFS – Depth-First Search)





Algoritmo DFS

Algoritmo DFS

- Utilizamos um vetor chamado **marca** de tamanho n que indica o estado do vértice v_i, onde marca$[v_i] =$

Algoritmo DFS

- Utilizamos um vetor chamado **marca** de tamanho n que indica o estado do vértice v_i, onde marca$[v_i] =$
  - branco; se v_i é ainda não visitado;

Algoritmo DFS

- Utilizamos um vetor chamado **marca** de tamanho n que indica o estado do vértice v_i, onde marca$[v_i] =$
  - branco; se v_i é ainda não visitado;
  - cinza; se v_i foi visitado, mas não percorrido;

Algoritmo DFS

- Utilizamos um vetor chamado **marca** de tamanho n que indica o estado do vértice v_i, onde marca$[v_i] =$
  - branco; se v_i é ainda não visitado;
  - cinza; se v_i foi visitado, mas não percorrido;
  - preto; se v_i foi completamente percorrido.

Algoritmo DFS

- Utilizamos um vetor chamado **marca** de tamanho n que indica o estado do vértice v_i, onde marca$[v_i] =$
  - branco; se v_i é ainda não visitado;
  - cinza; se v_i foi visitado, mas não percorrido;
  - preto; se v_i foi completamente percorrido.
- O algoritmo utiliza um procedimento auxiliar chamado **Visita** e uma pilha P.

Algoritmo 1: DFS(G, v)

Entrada: Grafo $G = (V, E)$ definido por listas de adjacências e vértice $v \in V$.

```
1 para i ← 1, ..., n faça
2     marca[i] ← branco;
3 Inicia pilha P vazia;
4 marca[v] ← cinza;
5 Visita(G, v);
```

Procedimento Visita

Algoritmo 2: Visita(G, v)

Entrada: Grafo $G = (V, E)$ definido por listas de adjacências e vértice $v \in V$.

```
1 Insere v em P;
2 u ← G[v].prox;
3 enquanto u ≠ λ faça
4 |   se marca[u] = branco então
5 |   |   marca[u] = cinza;
6 |   |   Visita(G, u);
7 Remove v de P;
8 marca[v] ← preto;
```

Complexidade

Complexidade

- A complexidade de DFS é dado pelo procedimento Visita.

Complexidade

- A complexidade de DFS é dado pelo procedimento Visita.
- Cada vértice v é inserido e removido exatamente uma vez em P: quando marca[v]=branco e quando marca[v]=preto.

Complexidade

- A complexidade de DFS é dado pelo procedimento Visita.
- Cada vértice v é inserido e removido exatamente uma vez em P: quando marca[v]=branco e quando marca[v]=preto.
- Enquanto marca[v]=cinza, percorre-se todas as arestas incidentes a v, a menos da aresta de árvore da busca.

Complexidade

- A complexidade de DFS é dado pelo procedimento Visita.
- Cada vértice v é inserido e removido exatamente uma vez em P: quando marca[v]=branco e quando marca[v]=preto.
- Enquanto marca[v]=cinza, percorre-se todas as arestas incidentes a v, a menos da aresta de árvore da busca.
- Cada aresta $uv \in E(G)$ é percorrida no máximo duas vezes.

Complexidade

- A complexidade de DFS é dado pelo procedimento Visita.
- Cada vértice v é inserido e removido exatamente uma vez em P: quando marca[v]=branco e quando marca[v]=preto.
- Enquanto marca[v]=cinza, percorre-se todas as arestas incidentes a v, a menos da aresta de árvore da busca.
- Cada aresta $uv \in E(G)$ é percorrida no máximo duas vezes.
- Logo a complexidade do algoritmo é $O(n + m)$.

Sumário

1. Introdução
2. Digrafos Acíclicos
3. Busca em Profundidade – DFS
4. Ordenação Topológica

Encontrando uma Ordenação Topológica

- Dado um DAG $G = (V, E)$, utilizamos buscas em profundidade sucessivas de modo que todos os vértices passem a ser pretos.

Encontrando uma Ordenação Topológica

- Dado um DAG $G = (V, E)$, utilizamos buscas em profundidade sucessivas de modo que todos os vértices passem a ser pretos.
- A partir de um vértice v, efetuamos uma DFS com origem em v.

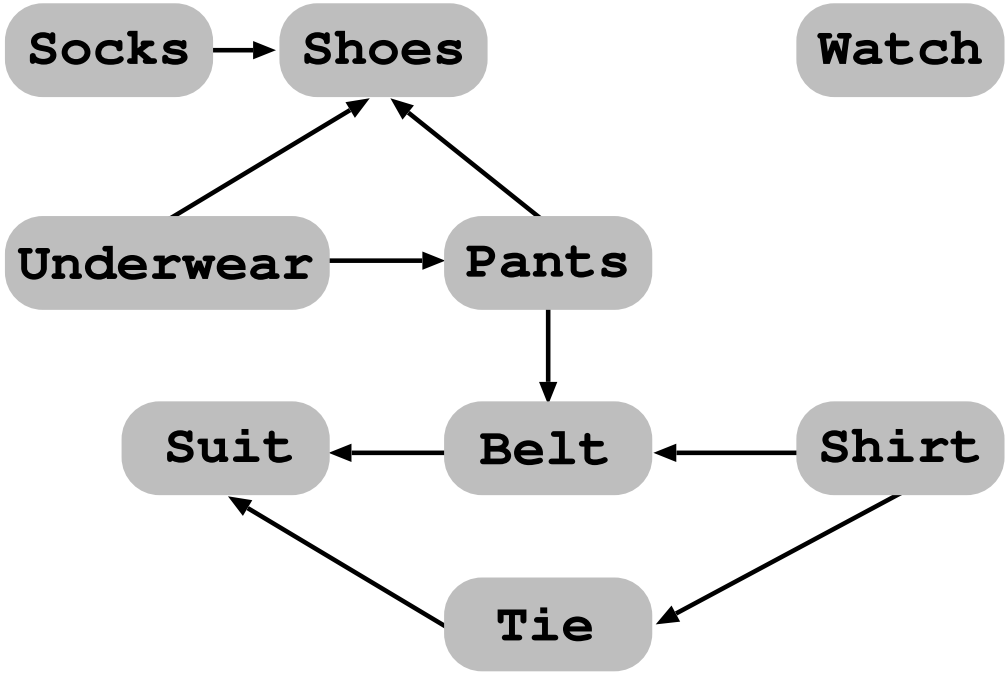
Encontrando uma Ordenação Topológica

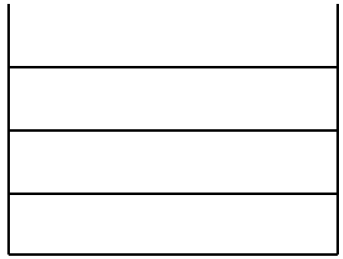
- Dado um DAG $G = (V, E)$, utilizamos buscas em profundidade sucessivas de modo que todos os vértices passem a ser pretos.
- A partir de um vértice v, efetuamos uma DFS com origem em v.
- Inserimos em uma lista encadeada L os nós que se tornam negros, ou seja, quando estes não possuem vizinhos brancos.

Encontrando uma Ordenação Topológica

- Dado um DAG $G = (V, E)$, utilizamos buscas em profundidade sucessivas de modo que todos os vértices passem a ser pretos.
- A partir de um vértice v, efetuamos uma DFS com origem em v.
- Inserimos em uma lista encadeada L os nós que se tornam negros, ou seja, quando estes não possuem vizinhos brancos.
- Logo, estes vértices são inseridos no começo de L.

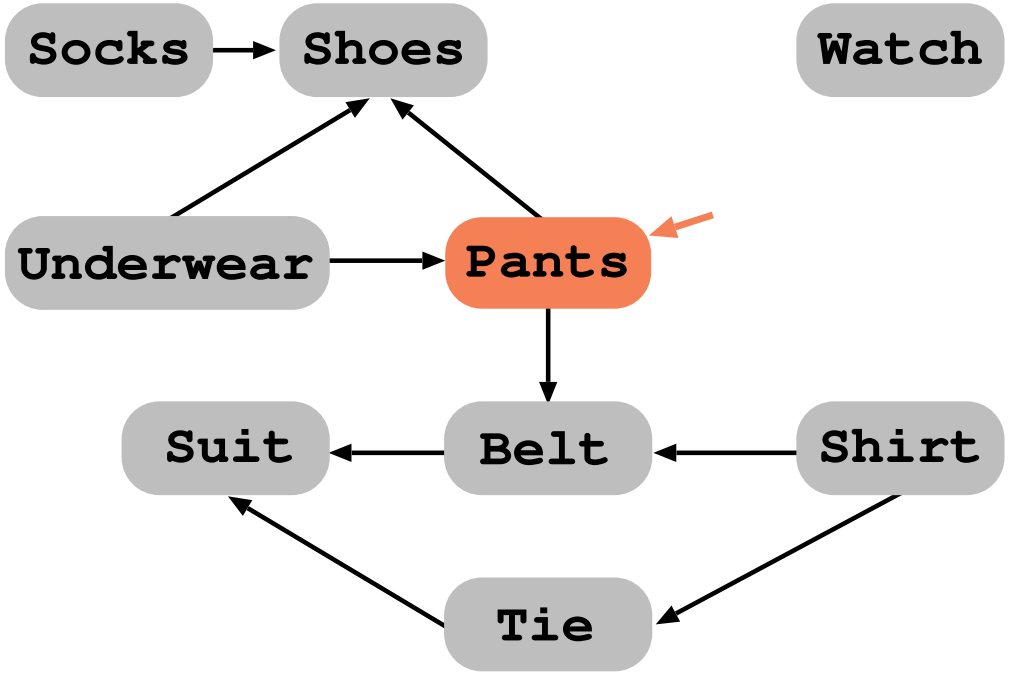
Ordenação Topológica

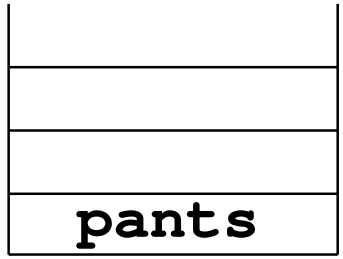




$\texttt{pt} \rightarrow \lambda$

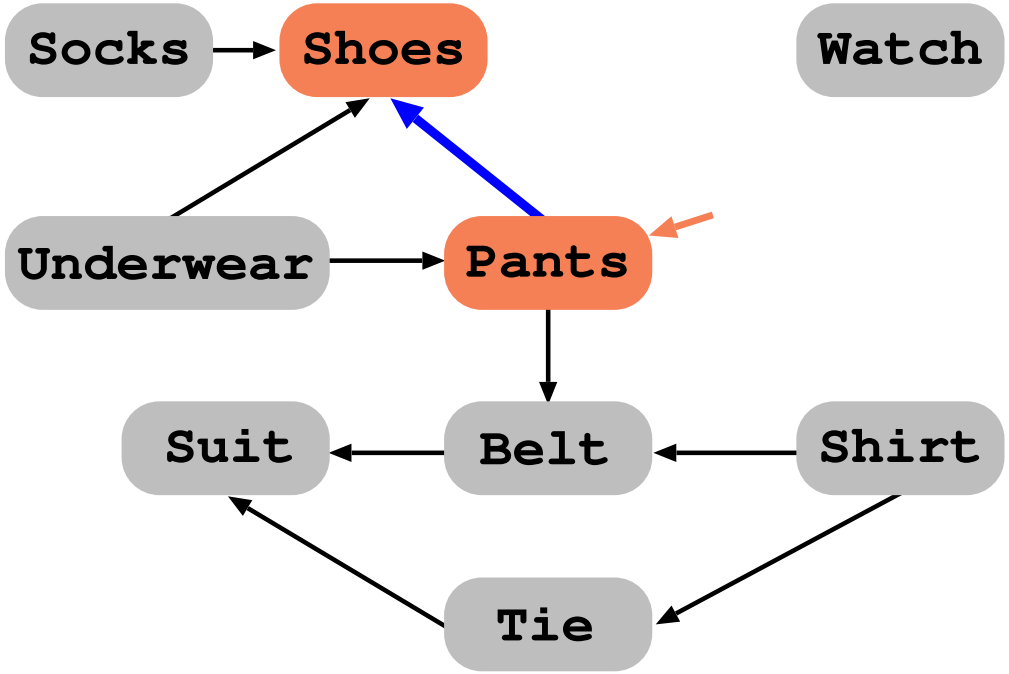
Ordenação Topológica

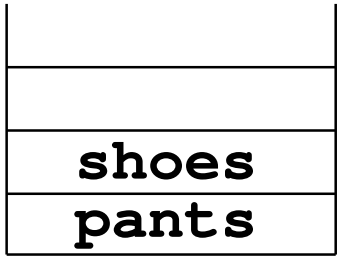




$pt \rightarrow \lambda$

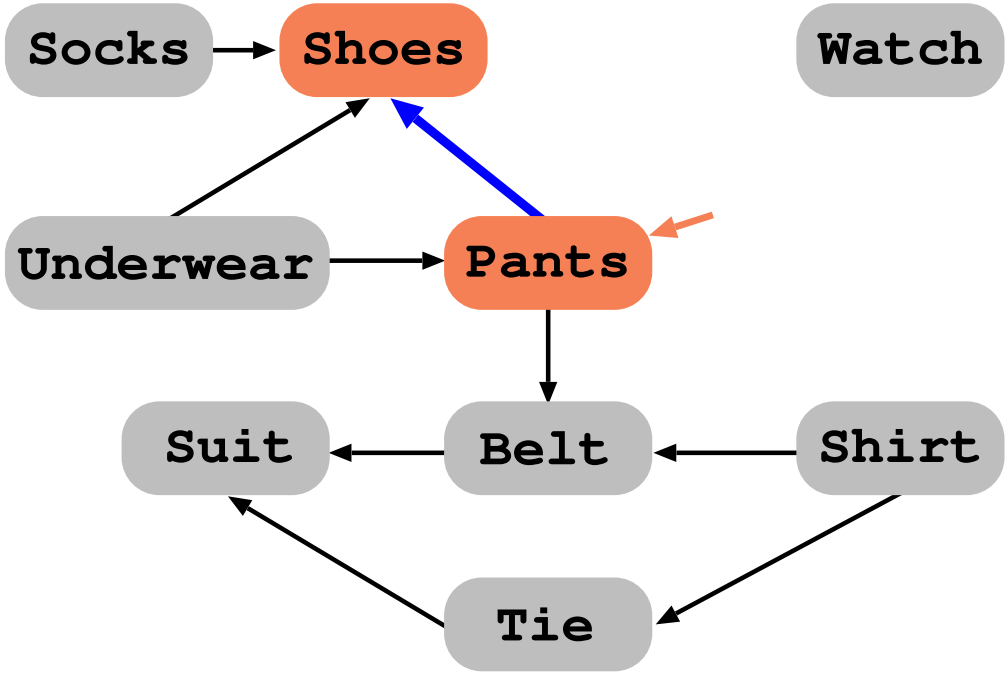
Ordenação Topológica

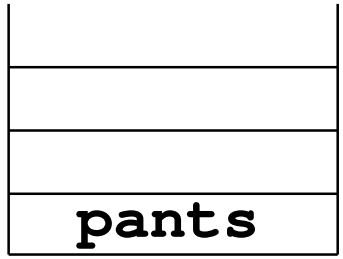




pt → λ

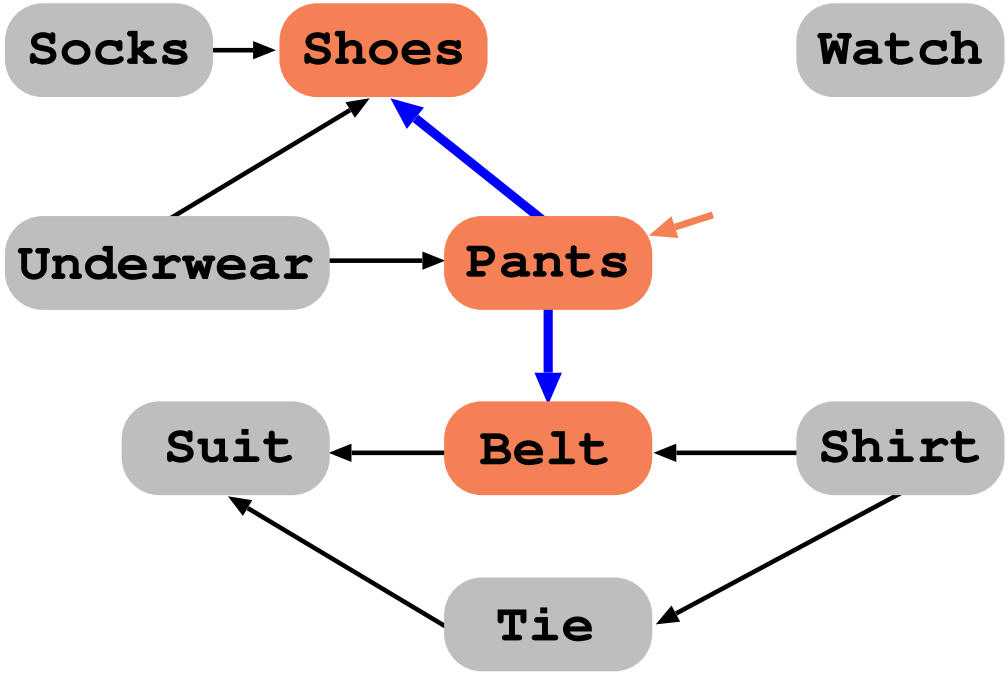
Ordenação Topológica

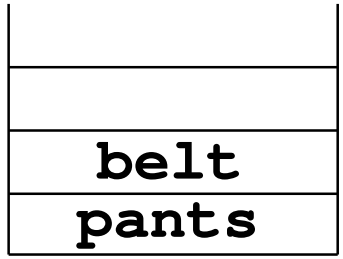






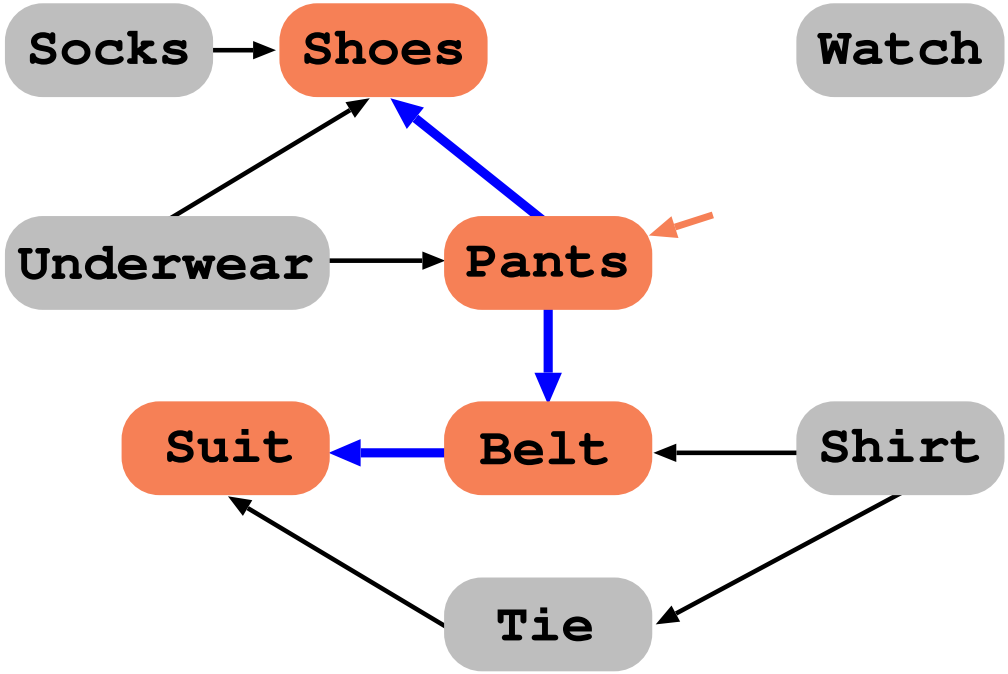
Ordenação Topológica

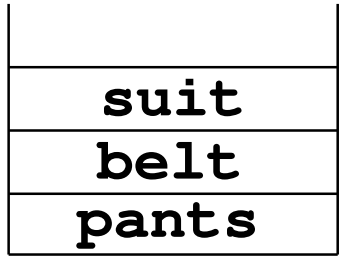






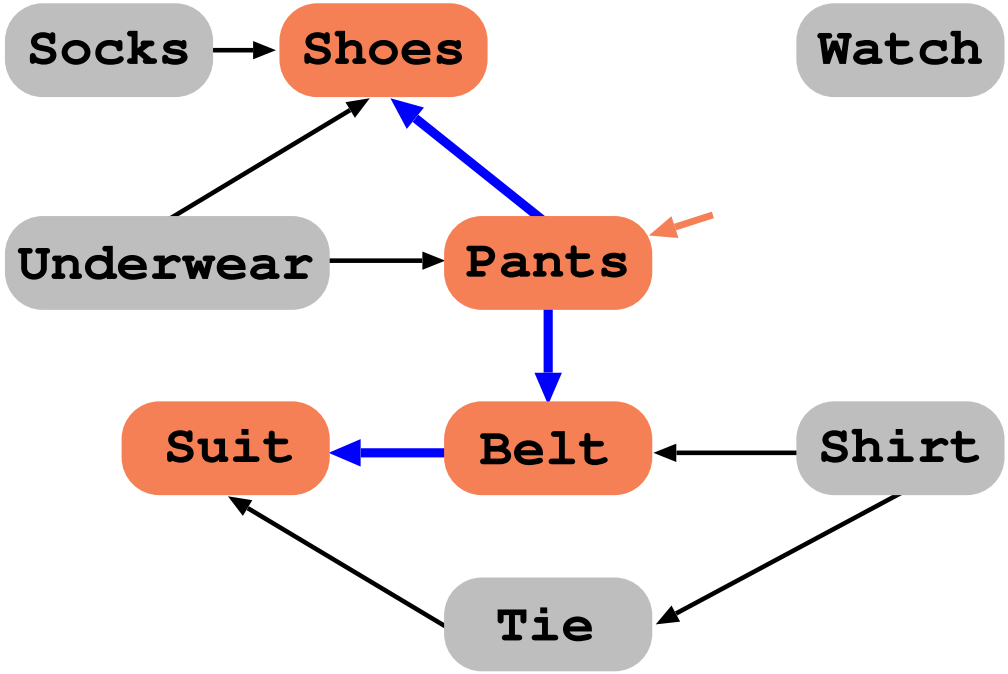
Ordenação Topológica

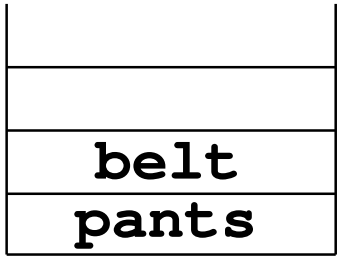


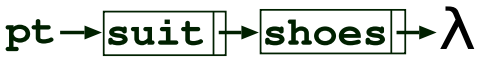




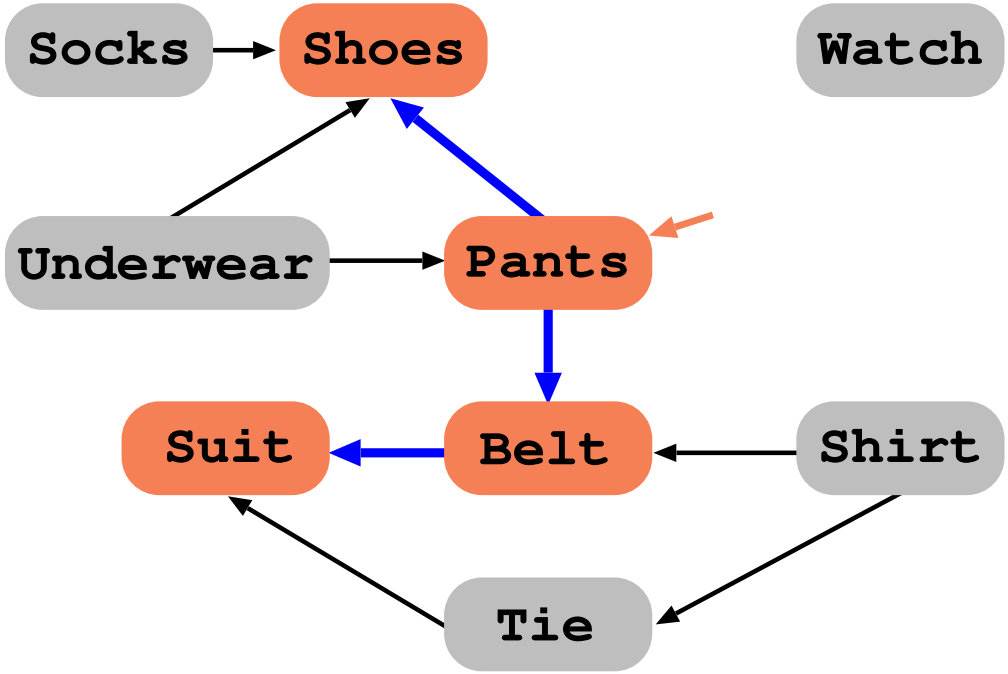
Ordenação Topológica

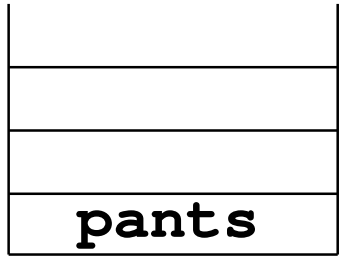






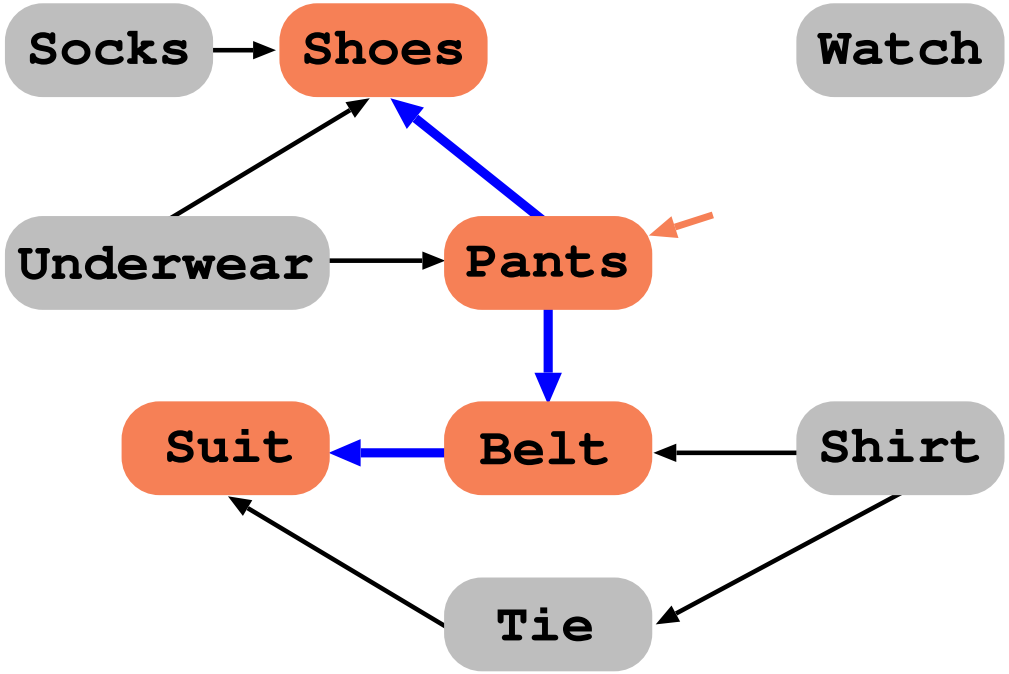
Ordenação Topológica

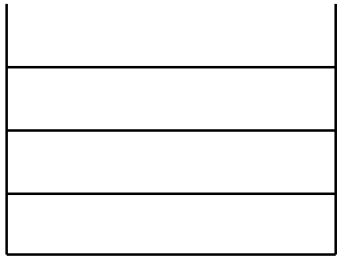


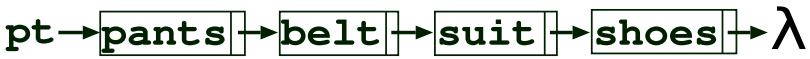




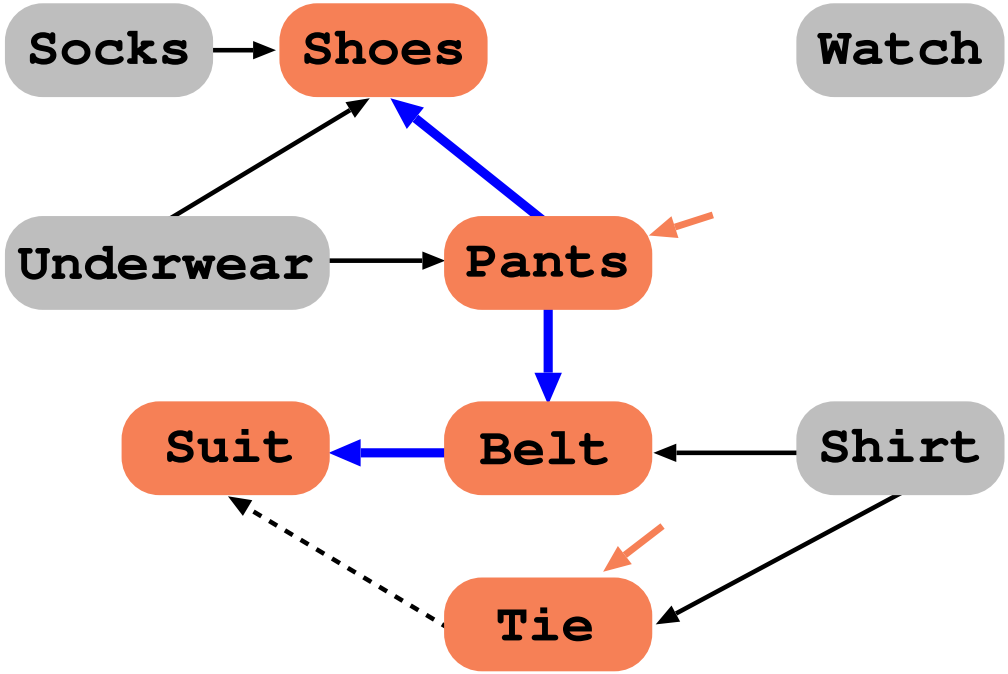
Ordenação Topológica

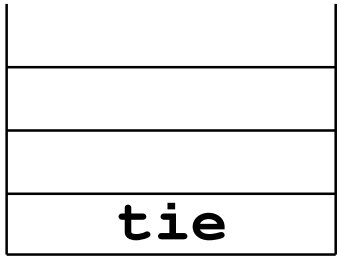


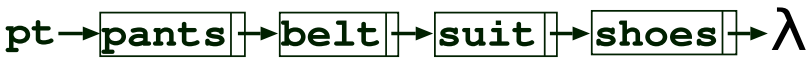




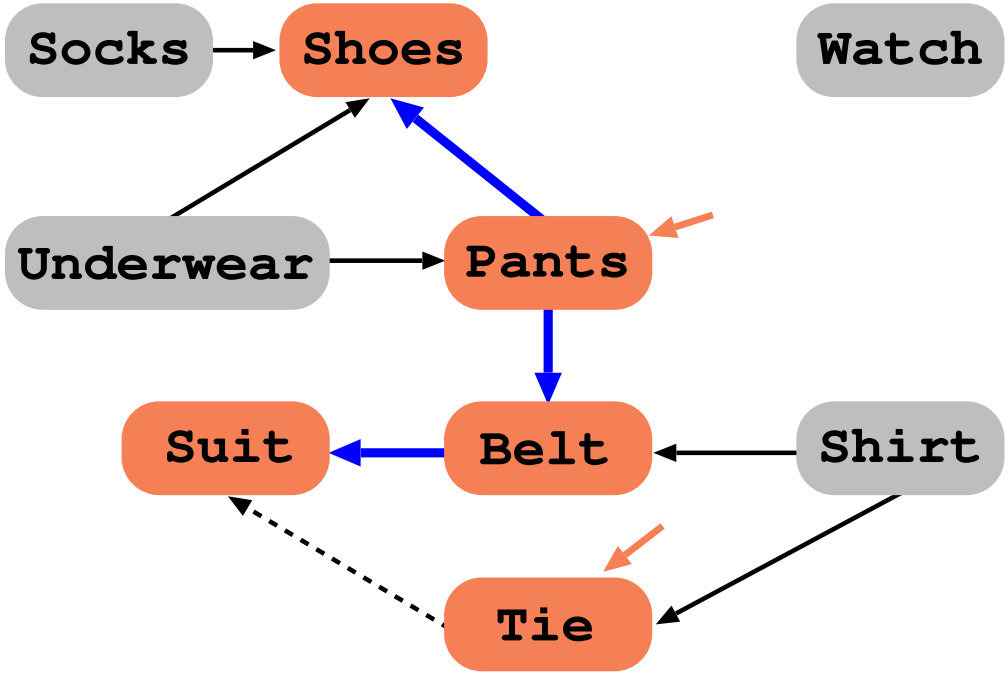
Ordenação Topológica

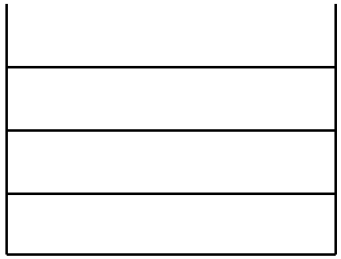






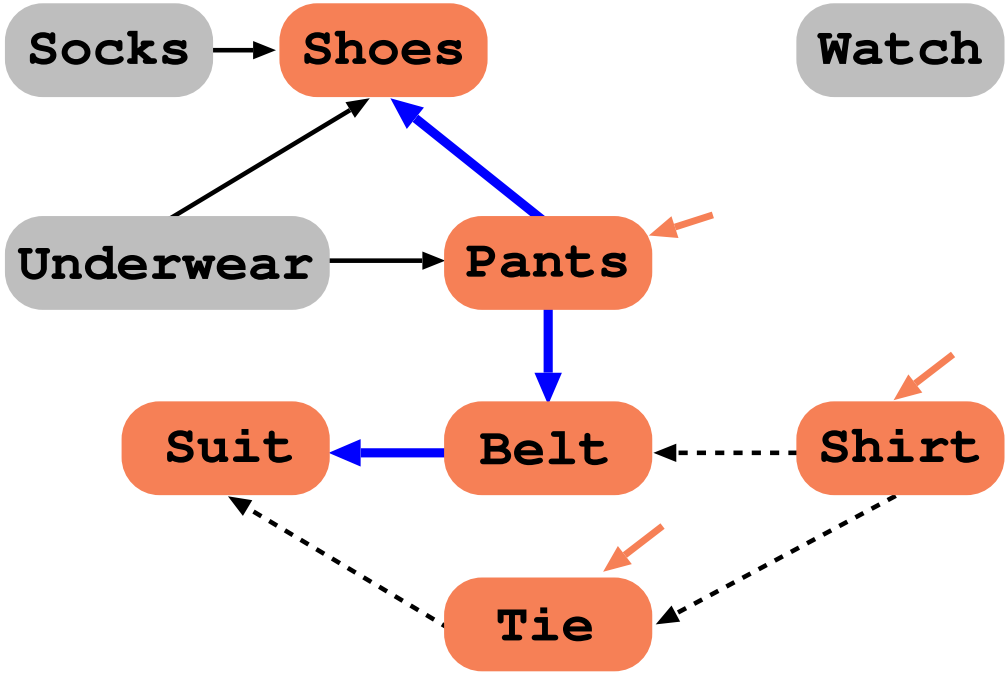
Ordenação Topológica

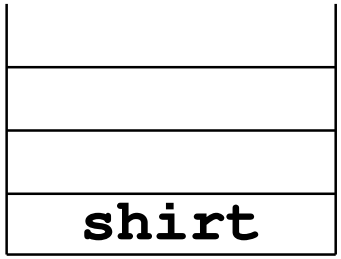






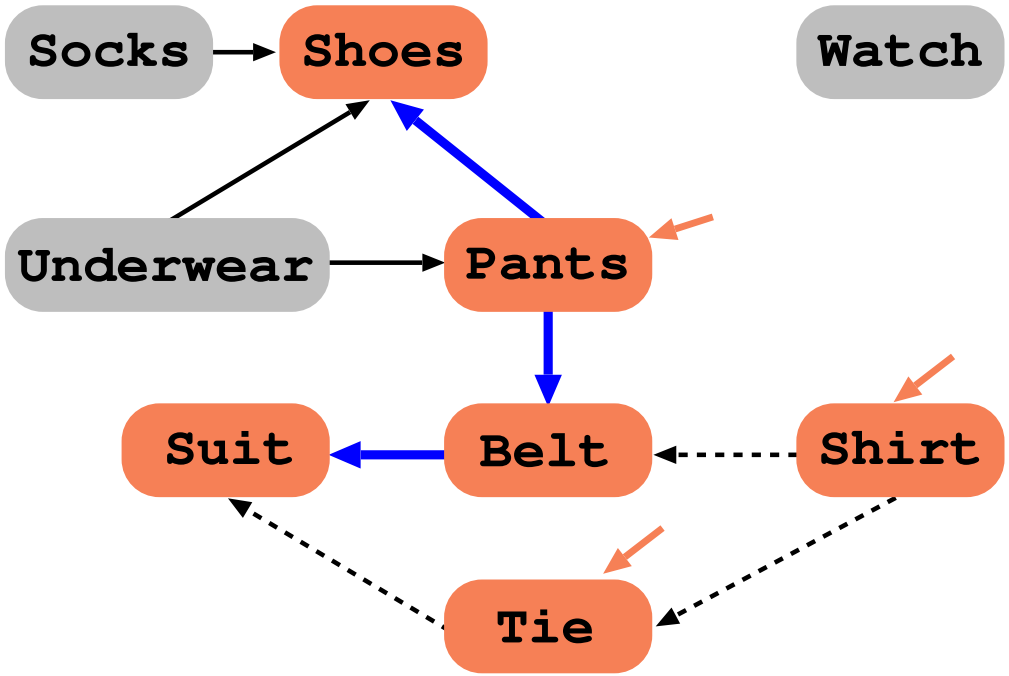
Ordenação Topológica

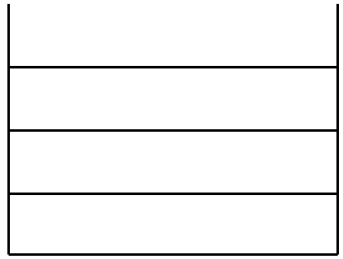






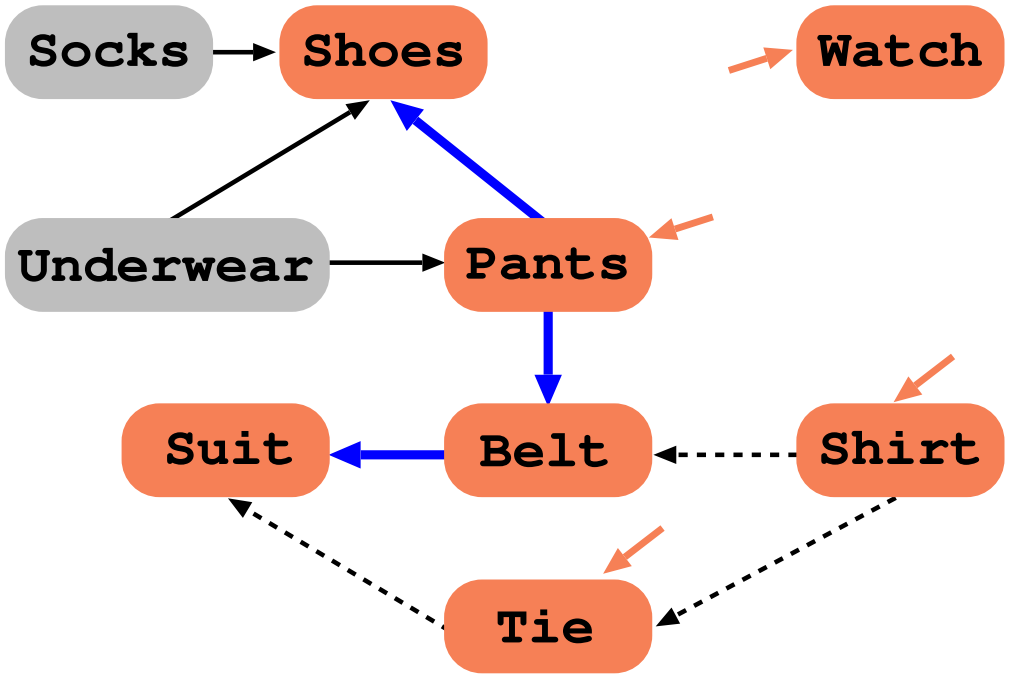
Ordenação Topológica

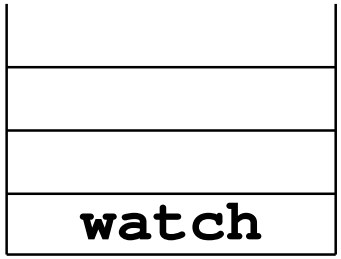


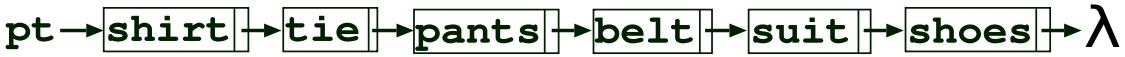




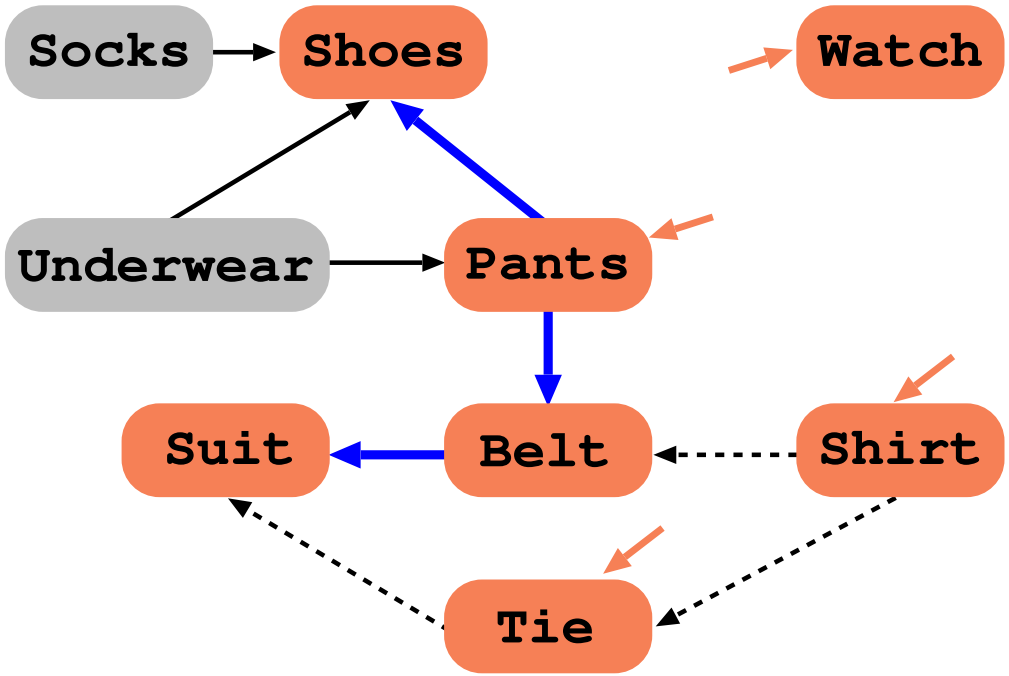
Ordenação Topológica

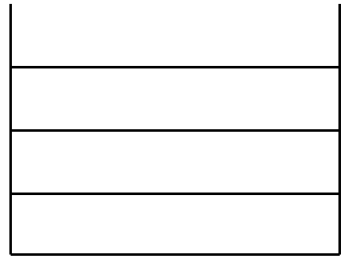






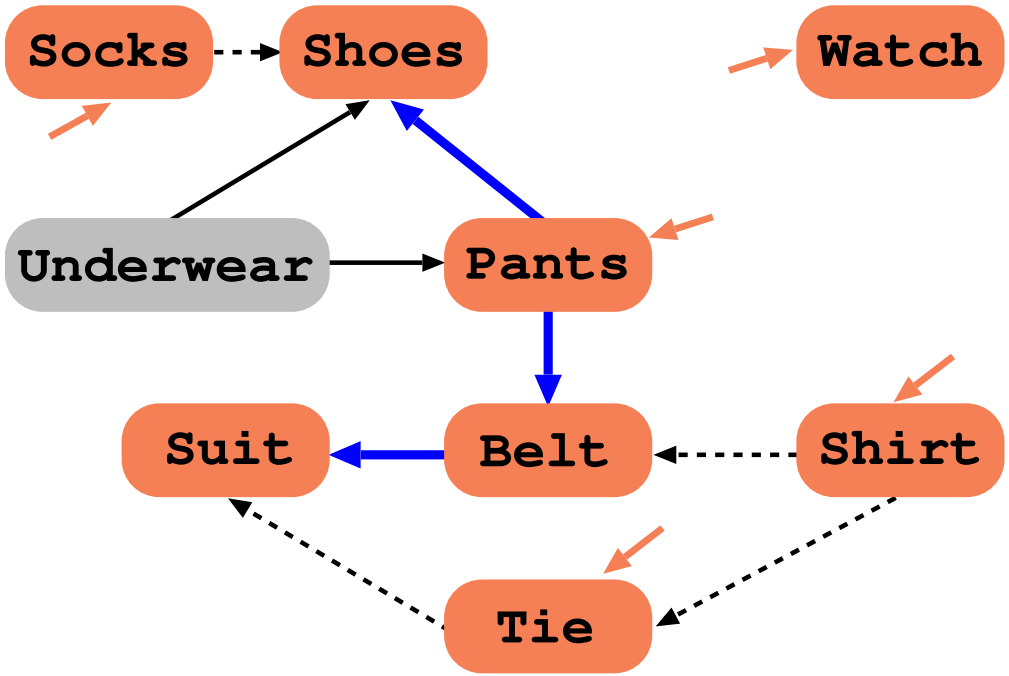
Ordenação Topológica

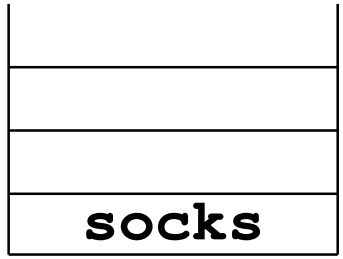






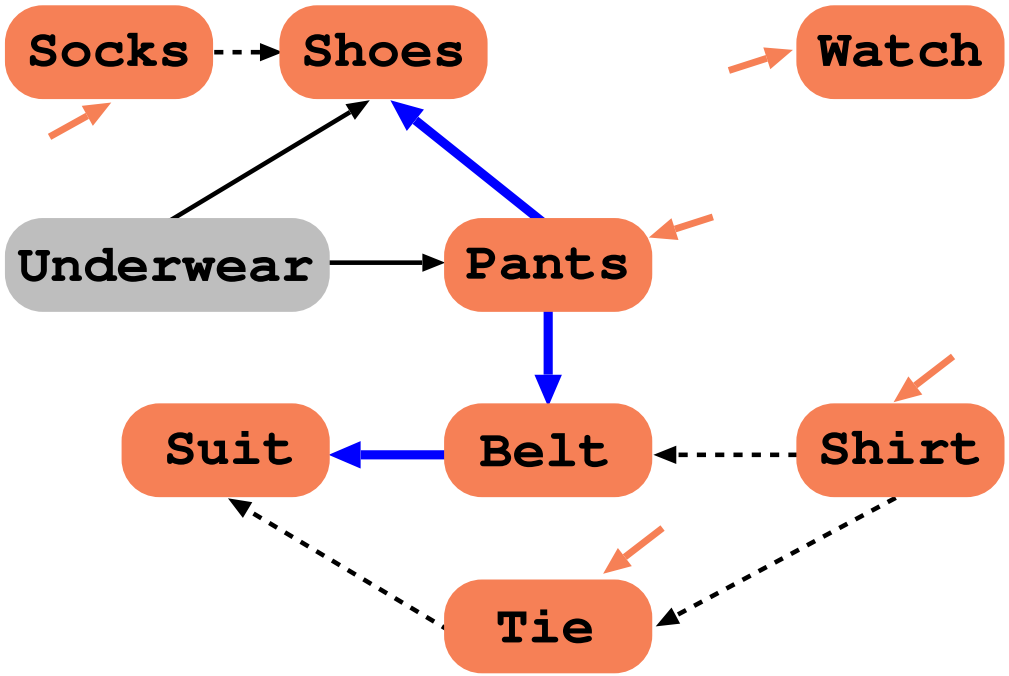
Ordenação Topológica

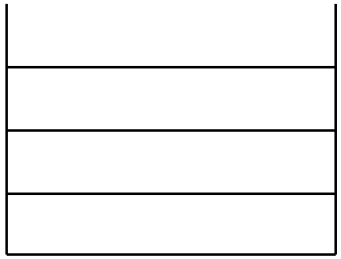


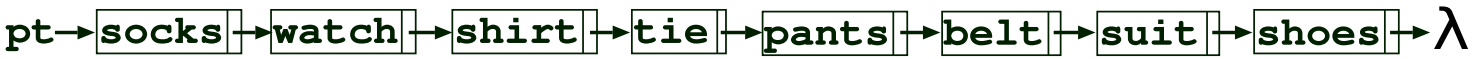




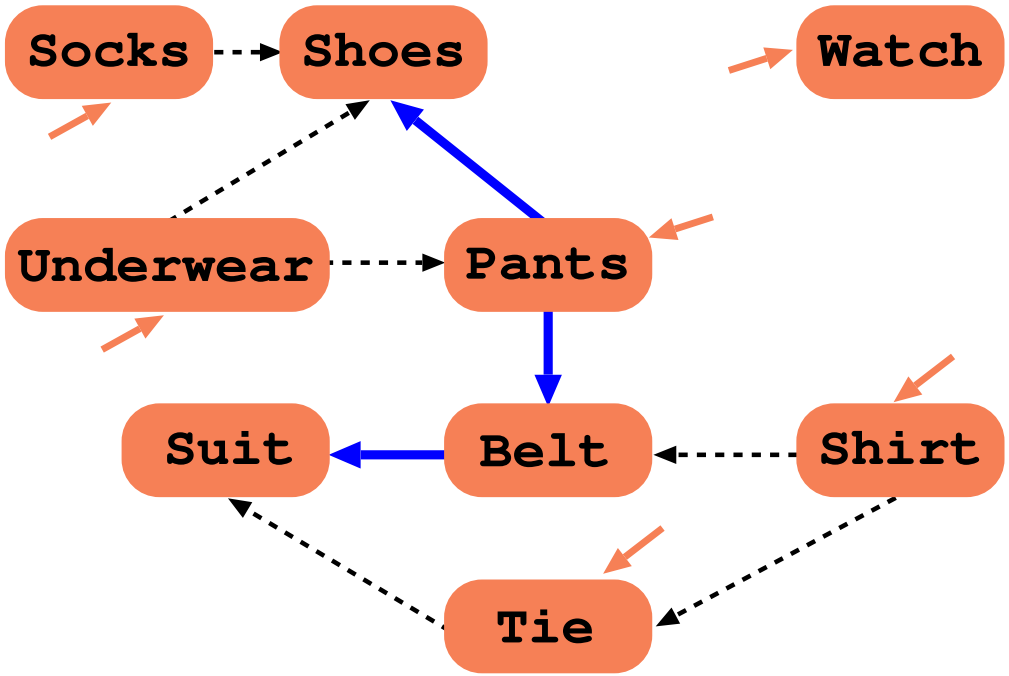
Ordenação Topológica

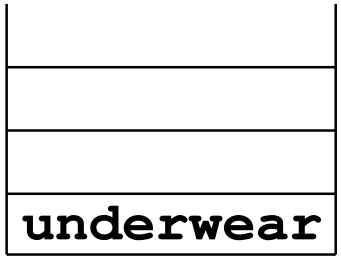


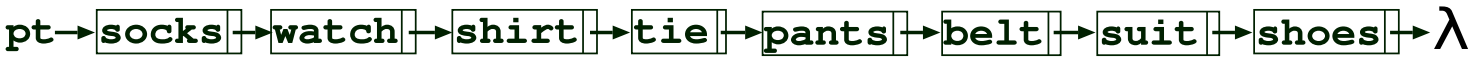




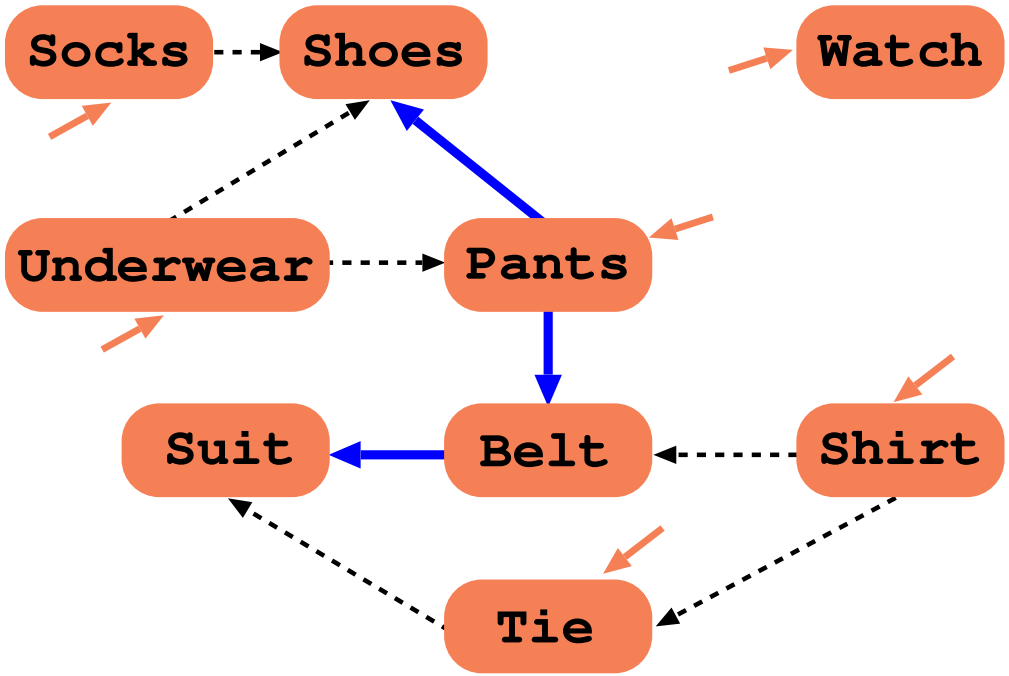
Ordenação Topológica

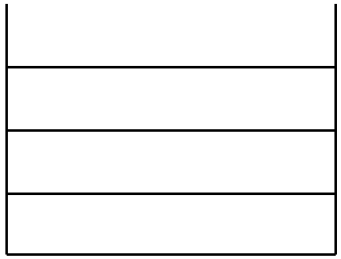






Ordenação Topológica







Algoritmo Ordenação Topológica

Algoritmo 3: OrdTop(G, v)

Entrada: Grafo $G = (V, E)$ definido por listas de adjacências e vértice $v \in V$.

```
1 para i ← 1, ..., n faça
2     marca[i] ← branco;
3 Inicia Lista encadeada L vazia;
4 Inicia Pilha P vazia;
5 para i ← 1, ..., n faça
6     se marca[i] = branco;
7     então
8         DFS'(G, i, P, L)      % Insere i no começo de L ao ser removido
                                  de P.;
```

Algoritmo Ordenação Topológica

Algoritmo 4: OrdTop(G, v)

Entrada: Grafo $G = (V, E)$ definido por listas de adjacências e vértice $v \in V$.

```
1 para i ← 1, ..., n faça
2     marca[i] ← branco;
3 Inicia Lista encadeada L vazia;
4 Inicia Pilha P vazia;
5 para i ← 1, ..., n faça
6     se marca[i] = branco;
7     então
8         DFS'(G, i, P, L)      % Insere i no começo de L ao ser removido
              de P.;
```

- A complexidade é análoga ao do percurso em profundidade, pois cada vértice é inserido e removido exatamente uma vez da pilha.